23CS11E - WEBFRAMEWORK USING PYTHON

Micro Project

Report On

Student Skill Exchange Platform

Submitted by

BALAGURUNATHAN H 2403957610421008

In partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



NATIONAL ENGINEERING COLLEGE
(An Autonomous Institution affiliated to Anna University, Chennai)

K.R.NAGAR, KOVILPATTI - 628503

OCTOBER – 2026

NATIONAL ENGINEERING COLLEGE

K.R.NAGAR, KOVILPATTI – 628503

BONAFIDE CERTIFICATE

This is to certify that this project report is a Bonafide record of **BALAGURUNATHAN H (2403957610421008)** work done for the course **23CS11E - WEBFRAMEWORK USING PYTHON** at the NATIONAL ENGINEERING COLLEGE, K.R. NAGAR, during the academic year 2026 – 2027 who carried out the project work under my supervision.

Course Instructor/Guide

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO
1	INTRODUCTION	5
2	IMPLEMENTATION	7
3	IMPLEMENTATION RESULTS	40
4	CONCLUSION	46

ABSTRACT

The Student Skill Exchange Platform is a web-based application that lets college students share what they know and learn from their classmates through one-to-one peer mentoring. Students can register with their college details, publish the skills they are willing to teach, discover skills offered by others, and send learning requests to the mentors they choose.

The system is organised into six modules: Authentication and Session Management, Student Profile Management, Skills Marketplace with search and filters, Peer Request Workflow (send, accept, decline, complete), Ratings and Reviews, and a Course Materials Hub for sharing PDFs, videos and links. A personalised dashboard summarises each student's skills, requests and completed sessions.

The application is built with Python Flask on the server, Jinja2 templates for rendering, MongoDB (through PyMongo) for storage, and hand-written HTML5 and CSS3 for the interface. Passwords are stored as salted hashes using Werkzeug, access to protected pages is controlled with Flask sessions, and ownership checks make sure students can only modify their own skills, requests and materials.

By replacing scattered chat groups and word-of-mouth help with a single searchable platform, the Student Skill Exchange Platform makes peer learning easier to find, easier to organise and easier to measure on campus.

CHAPTER 1
INTRODUCTION

1.1 INTRODUCTION

Students on a campus hold a wide range of skills, from programming and web development to design, public speaking and music. Much of this knowledge is shared informally, and only among close friends. A student who needs help with Flask, or who could teach Python to juniors, usually has no easy way to find the right person.

The Student Skill Exchange Platform was developed to solve this. It is a Flask web application where every student can list the skills they can teach, browse what others offer, and request a one-to-one session. Mentors accept or decline requests, mark sessions as completed, and receive ratings and written reviews from the students they helped.

The platform also acts as a small learning hub. Each skill page carries a Course Materials section where mentors and peers upload PDFs, lecture notes, lab exercises and video links, preview them inside the browser, and mark the useful ones. Together these features turn peer learning into an organised, visible and rewarding activity.

1.2 PROBLEM STATEMENT

In most colleges, peer learning depends on personal contacts. Students do not know who is strong in which topic, requests for help are scattered across messaging groups, and good answers are lost once a chat scrolls away. Mentors have no way to show what they can teach, and learners have no way to judge whether a mentor is helpful.

There is also no record of what actually happened. Requests go unanswered, sessions are not tracked, study materials circulate as untraceable files, and feedback is never collected. A student who regularly helps others receives no visible recognition for it. There is a clear need for a single, structured platform that lists skills, manages the request-and-response workflow, keeps learning materials in one place, and records feedback.

1.3 OBJECTIVES

The primary objective of the Student Skill Exchange Platform is to provide one central place where students can offer skills, find mentors and arrange peer learning sessions with minimum effort. It must offer

simple registration and login, editable student profiles, and full create, view, update and delete control over skill listings.

The system aims to support the complete learning workflow: a student sends a request, the mentor accepts or declines it, the session is marked completed, and the learner leaves a one-to-five star rating with a comment. Duplicate and self-requests are blocked, and a session can be reviewed only once.

Further objectives are to let students search and filter skills by keyword, category and level; to share PDFs, documents and video links inside each skill with in-browser preview, download counts and helpful votes; to give each student a dashboard of key statistics; and to protect user data through hashed passwords, session-based access control and ownership checks.

CHAPTER 2
IMPLEMENTATION

2.1 PROJECT STRUCTURE

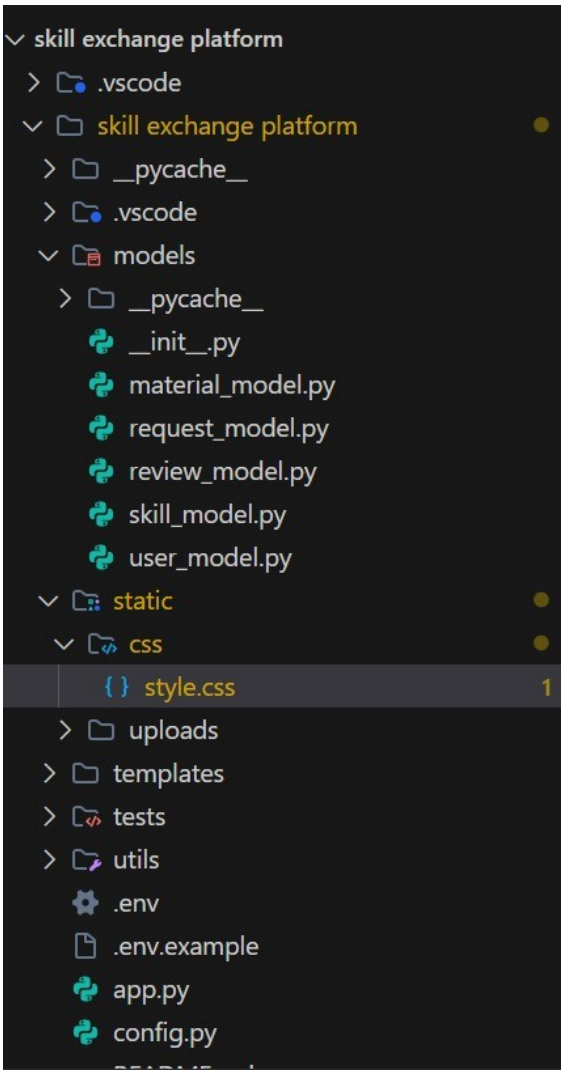


Figure 2.1: Project Folder Structure

2.2 PROJECT CODE

app.py:

```
import sys
import os
from flask import Flask, render_template, request, redirect, url_for, session, flash, abort,
send_from_directory, jsonify
from werkzeug.utils import secure_filename
from pymongo import MongoClient
from bson import ObjectId
from datetime import datetime, timezone

from config import Config
from models.user_model import UserModel
```

```
from models.skill_model import SkillModel
from models.request_model import RequestModel
from models.review_model import ReviewModel
from models.material_model import MaterialModel
from utils.helpers import (
    login_required,
    parse_object_id,
    format_datetime,
    time_ago,
    format_file_size,
    get_youtube_embed_info,
    detect_material_type,
    init_db_indexes,
    seed_sample_data
)

# Initialize Flask application
app = Flask(__name__)
app.config.from_object(Config)

# Ensure upload directory exists
os.makedirs(app.config.get("UPLOAD_FOLDER", os.path.join(os.path.dirname(os.path.abspath(__file__)),
'static', 'uploads', 'materials')), exist_ok=True)

# Register custom Jinja2 template filters
app.jinja_env.filters['format_date'] = format_datetime
app.jinja_env.filters['time_ago'] = time_ago
app.jinja_env.filters['file_size'] = format_file_size

# Connect to MongoDB
try:
    client = MongoClient(app.config["MONGO_URI"], serverSelectionTimeoutMS=5000)
    db = client[app.config["DATABASE_NAME"]]
    # Ensure indexes are created
    init_db_indexes(db)
    print(f"[OK] Connected to MongoDB database: {app.config['DATABASE_NAME']}")
except Exception as e:
    print(f"[WARNING] MongoDB connection warning: {e}")
    db = None

# Initialize Models
user_model = UserModel(db) if db is not None else None
skill_model = SkillModel(db) if db is not None else None
request_model = RequestModel(db) if db is not None else None
review_model = ReviewModel(db) if db is not None else None
material_model = MaterialModel(db) if db is not None else None


@app.context_processor
def inject_globals():
    """
    Inject global configuration lists and session data into all templates.
    """
    return {
        "categories": Config.CATEGORIES,
        "skill_levels": Config.SKILL_LEVELS,
        "departments": Config.DEPARTMENTS,
        "years": Config.YEARS,
        "resource_tags": getattr(Config, 'RESOURCE_TAGS', [
            "Lecture Notes", "Video Tutorial", "Cheat Sheet", "Lab Exercise", "Slide Deck", "Reference
Code", "Other"
        ]),
        "now": datetime.now(timezone.utc)
    }


# ========================================
```

```
# Authentication & Landing Routes
# ========================================

@app.route('/')
def index():
    """
    Homepage showcasing platform highlights, features, and recent skills.
    """
    recent_skills = skill_model.get_recent_skills(limit=6) if skill_model else []
    return render_template('index.html', recent_skills=recent_skills)


@app.route('/register', methods=['GET', 'POST'])
def register():
    """
    Register a new college student account.
    """
    if 'user_id' in session:
        return redirect(url_for('dashboard'))

    if request.method == 'POST':
        name = request.form.get('name', '').strip()
        email = request.form.get('email', '').strip().lower()
        password = request.form.get('password', '').strip()
        department = request.form.get('department', '').strip()
        year = request.form.get('year', '').strip()
        bio = request.form.get('bio', '').strip()

        # Validation
        if not name or not email or not password or not department or not year:
            flash('All required fields must be filled in.', 'danger')
            return render_template('register.html')

        if len(password) < 6:
            flash('Password must be at least 6 characters long.', 'danger')
            return render_template('register.html')

        user, err = user_model.create_user(
            name=name,
            email=email,
            password=password,
            department=department,
            year=year,
            bio=bio
        )

        if err:
            flash(err, 'danger')
            return render_template('register.html')

        # Create session on successful registration
        session['user_id'] = str(user['_id'])
        session['user_name'] = user['name']
        session['user_email'] = user['email']
        session['user_image'] = user.get('profile_image', '')

        flash('Registration successful! Welcome to the Skill Sharing Platform.', 'success')
        return redirect(url_for('dashboard'))

    return render_template('register.html')


@app.route('/login', methods=['GET', 'POST'])
def login():
    """
    Log in an existing student using email and password.
    """
```

```
    if 'user_id' in session:
        return redirect(url_for('dashboard'))

    if request.method == 'POST':
        email = request.form.get('email', '').strip().lower()
        password = request.form.get('password', '').strip()
        next_url = request.args.get('next')

        if not email or not password:
            flash('Please provide both email and password.', 'danger')
            return render_template('login.html')

        user = user_model.get_by_email(email)
        if not user or not user_model.verify_password(user, password):
            flash('Invalid email or password.', 'danger')
            return render_template('login.html')

        # Store user details in Flask session
        session['user_id'] = str(user['_id'])
        session['user_name'] = user['name']
        session['user_email'] = user['email']
        session['user_image'] = user.get('profile_image', '')

        flash(f'Welcome back, {user["name"]}!', 'success')
        if next_url:
            return redirect(next_url)
        return redirect(url_for('dashboard'))

    return render_template('login.html')


@app.route('/logout')
def logout():
    """
    Clear session and log out.
    """
    session.clear()
    flash('You have been logged out successfully.', 'info')
    return redirect(url_for('login'))


# ========================================
# Dashboard & Profile Routes
# ========================================

@app.route('/dashboard')
@login_required
def dashboard():
    """
    Student dashboard with metrics, personal skills, and recent requests.
    """
    user_id = session.get('user_id')
    user = user_model.get_by_id(user_id)
    if not user:
        session.clear()
        flash('User session expired. Please log in again.', 'warning')
        return redirect(url_for('login'))

    my_skills = skill_model.get_by_owner_id(user_id)
    stats = request_model.get_user_stats(user_id)

    # Fetch recent requests sent or received
    sent_reqs = request_model.get_sent_requests(user_id)[:4]
    received_reqs = request_model.get_received_requests(user_id)[:4]
    all_recent_requests = sorted(sent_reqs + received_reqs, key=lambda r: r.get('created_at',
datetime.min), reverse=True)[:5]
```

```
    # Recent community skills excluding self
    all_recent = skill_model.get_recent_skills(limit=6)
    recent_community_skills = [s for s in all_recent if str(s.get('owner_id')) != user_id][:3]

    # Shared materials by this student
    my_materials = material_model.get_by_uploader(user_id, limit=6) if material_model else []
    my_materials_count = material_model.count_by_uploader(user_id) if material_model else 0

    return render_template(
        'dashboard.html',
        user=user,
        my_skills=my_skills,
        stats=stats,
        recent_requests=all_recent_requests,
        recent_community_skills=recent_community_skills,
        my_materials=my_materials,
        my_materials_count=my_materials_count
    )


@app.route('/profile', methods=['GET', 'POST'])
@login_required
def profile():
    """
    View and edit personal student profile.
    """
    user_id = session.get('user_id')
    user = user_model.get_by_id(user_id)
    if not user:
        flash('User not found.', 'danger')
        return redirect(url_for('login'))

    if request.method == 'POST':
        name = request.form.get('name', '').strip()
        department = request.form.get('department', '').strip()
        year = request.form.get('year', '').strip()
        bio = request.form.get('bio', '').strip()
        profile_image = request.form.get('profile_image', '').strip()

        if not name or not department or not year:
            flash('Name, department, and year are required.', 'danger')
        else:
            success, err = user_model.update_profile(
                user_id=user_id,
                name=name,
                department=department,
                year=year,
                bio=bio,
                profile_image=profile_image
            )
            if success:
                session['user_name'] = name
                if profile_image:
                    session['user_image'] = profile_image
                flash('Profile updated successfully.', 'success')
                return redirect(url_for('profile'))
            else:
                flash(f'Failed to update profile: {err}', 'danger')

    my_skills = skill_model.get_by_owner_id(user_id)
    stats = request_model.get_user_stats(user_id)
    my_materials_count = material_model.count_by_uploader(user_id) if material_model else 0

    return render_template('profile.html', user=user, my_skills=my_skills, stats=stats,
my_materials_count=my_materials_count)
```

```
# ========================================
# Skills Management Routes
# ========================================

@app.route('/skills')
def skills():
    """
    Explore and search all skills with case-insensitive search and filters.
    """
    search_query = request.args.get('search', '').strip()
    category_filter = request.args.get('category', 'All').strip()
    level_filter = request.args.get('level', 'All').strip()

    skills_list = skill_model.get_all_skills(
        search=search_query if search_query else None,
        category=category_filter if category_filter != 'All' else None,
        level=level_filter if level_filter != 'All' else None
    )

    return render_template(
        'skills.html',
        skills=skills_list,
        current_search=search_query,
        current_category=category_filter,
        current_level=level_filter
    )


@app.route('/add-skill', methods=['GET', 'POST'])
@login_required
def add_skill():
    """
    Create a new skill offering with optional course PDF / material attachment.
    """
    if request.method == 'POST':
        title = request.form.get('title', '').strip()
        description = request.form.get('description', '').strip()
        category = request.form.get('category', '').strip()
        level = request.form.get('level', '').strip()
        owner_id = session.get('user_id')
        owner_name = session.get('user_name')

        if not title or not description or not category or not level:
            flash('All fields are required to offer a skill.', 'danger')
            return render_template('add_skill.html')

        skill, err = skill_model.create_skill(
            title=title,
            description=description,
            category=category,
            level=level,
            owner_id=owner_id,
            owner_name=owner_name
        )

        if err:
            flash(err, 'danger')
            return render_template('add_skill.html')

        # Update user's skills offered count
        user_model.increment_skills_offered(owner_id, 1)

        # Check if user attached an initial PDF or course material
        if 'material_file' in request.files:
            file = request.files['material_file']
            if file and file.filename != '':
                orig_filename = secure_filename(file.filename)
```

```
                ext = orig_filename.rsplit('.', 1)[-1].lower() if '.' in orig_filename else ''
                if ext in Config.ALLOWED_EXTENSIONS:
                    skill_dir = os.path.join(app.config['UPLOAD_FOLDER'], str(skill['_id']))
                    os.makedirs(skill_dir, exist_ok=True)
                    timestamp_str = datetime.now(timezone.utc).strftime("%Y%m%d_%H%M%S")
                    saved_filename = f"{timestamp_str}_{orig_filename}"
                    dest_path = os.path.join(skill_dir, saved_filename)
                    file.save(dest_path)
                    file_size_formatted = format_file_size(os.path.getsize(dest_path))
                    detected_type = detect_material_type(filename=orig_filename)
                    mat_title = request.form.get('material_title', '').strip() or f"{title} - Course
Notes"
                    mat_tag = request.form.get('material_tag', 'Lecture Notes').strip()

                    material_model.create_material(
                        skill_id=skill['_id'],
                        uploader_id=owner_id,
                        uploader_name=owner_name,
                        uploader_role="Course Mentor",
                        title=mat_title,
                        description="Initial course material uploaded with skill listing.",
                        material_type=detected_type,
                        file_path=f"{skill['_id']}/{saved_filename}",
                        file_name=file.filename,
                        file_size=file_size_formatted,
                        file_ext=ext,
                        tag=mat_tag
                    )

        flash('Skill added successfully! It is now visible to other students.', 'success')
        return redirect(url_for('skill_details', skill_id=skill['_id']))

    return render_template('add_skill.html')


@app.route('/edit-skill/<skill_id>', methods=['GET', 'POST'])
@login_required
def edit_skill(skill_id):
    """
    Edit an existing skill listing and manage course materials (Owner only).
    """
    skill = skill_model.get_by_id(skill_id)
    if not skill:
        flash('Skill not found.', 'danger')
        return redirect(url_for('skills'))

    user_id = session.get('user_id')
    owner_id = skill.get('owner_id') or (skill.get('mentor', {}).get('user_id') if
isinstance(skill.get('mentor'), dict) else None)
    if str(owner_id) != str(user_id):
        flash('Unauthorized: You can only edit your own skills.', 'danger')
        return redirect(url_for('skill_details', skill_id=skill_id))

    if request.method == 'POST':
        title = request.form.get('title', '').strip()
        description = request.form.get('description', '').strip()
        category = request.form.get('category', '').strip()
        level = request.form.get('level', '').strip()

        if not title or not description or not category or not level:
            flash('All fields are required.', 'danger')
            materials = material_model.get_by_skill_id(skill_id) if material_model else []
            return render_template('edit_skill.html', skill=skill, materials=materials)

        success, err = skill_model.update_skill(
            skill_id=skill_id,
            owner_id=user_id,
```

```
            title=title,
            description=description,
            category=category,
            level=level
        )

        # Check if user also attached a new PDF or course material while editing
        if 'material_file' in request.files:
            file = request.files['material_file']
            if file and file.filename != '':
                orig_filename = secure_filename(file.filename)
                ext = orig_filename.rsplit('.', 1)[-1].lower() if '.' in orig_filename else ''
                if ext in Config.ALLOWED_EXTENSIONS:
                    skill_dir = os.path.join(app.config['UPLOAD_FOLDER'], str(skill['_id']))
                    os.makedirs(skill_dir, exist_ok=True)
                    timestamp_str = datetime.now(timezone.utc).strftime("%Y%m%d_%H%M%S")
                    saved_filename = f"{timestamp_str}_{orig_filename}"
                    dest_path = os.path.join(skill_dir, saved_filename)
                    file.save(dest_path)
                    file_size_formatted = format_file_size(os.path.getsize(dest_path))
                    detected_type = detect_material_type(filename=orig_filename)
                    mat_title = request.form.get('material_title', '').strip() or f"{title} - Course
Notes"
                    mat_tag = request.form.get('material_tag', 'Lecture Notes').strip()

                    material_model.create_material(
                        skill_id=skill['_id'],
                        uploader_id=user_id,
                        uploader_name=session.get('user_name', 'Course Mentor'),
                        uploader_role="Course Mentor",
                        title=mat_title,
                        description="Course material uploaded during skill update.",
                        material_type=detected_type,
                        file_path=f"{skill['_id']}/{saved_filename}",
                        file_name=file.filename,
                        file_size=file_size_formatted,
                        file_ext=ext,
                        tag=mat_tag
                    )
                    flash('New course material attached successfully!', 'info')

        if success:
            flash('Skill updated successfully.', 'success')
            return redirect(url_for('skill_details', skill_id=skill_id))
        else:
            flash(f'Failed to update skill: {err}', 'danger')

    materials = material_model.get_by_skill_id(skill_id) if material_model else []
    return render_template('edit_skill.html', skill=skill, materials=materials)


@app.route('/delete-skill/<skill_id>', methods=['POST'])
@login_required
def delete_skill(skill_id):
    """
    Delete a skill listing (Owner only).
    """
    user_id = session.get('user_id')
    success, err = skill_model.delete_skill(skill_id, user_id)

    if success:
        user_model.increment_skills_offered(user_id, -1)
        flash('Skill deleted successfully.', 'success')
        return redirect(url_for('dashboard'))
    else:
        flash(f'Cannot delete skill: {err}', 'danger')
        return redirect(url_for('skills'))
```

```
@app.route('/skill/<skill_id>')
def skill_details(skill_id):
    """
    Display comprehensive skill details, mentor info, ratings, reviews, and course materials.
    """
    skill = skill_model.get_by_id(skill_id)
    if not skill:
        flash('Skill not found or has been removed.', 'warning')
        return redirect(url_for('skills'))

    owner_id = skill.get('owner_id') or (skill.get('mentor', {}).get('user_id') if
isinstance(skill.get('mentor'), dict) else None)
    owner = user_model.get_by_id(owner_id) if owner_id else None
    if not owner:
        owner_name = skill.get('owner_name') or (skill.get('mentor', {}).get('name') if
isinstance(skill.get('mentor'), dict) else "Student Mentor")
        owner_dept = skill.get('mentor', {}).get('department', "Student") if
isinstance(skill.get('mentor'), dict) else "Student"
        owner_yr = skill.get('mentor', {}).get('year', "Campus Peer") if
isinstance(skill.get('mentor'), dict) else "Campus Peer"
        owner = {"name": owner_name, "department": owner_dept, "year": owner_yr}

    reviews = review_model.get_by_skill_id(skill_id)
    review_stats = review_model.get_skill_stats(skill_id)

    # Fetch Course Materials (PDFs, Videos, Lab Exercises, Links)
    materials = material_model.get_by_skill_id(skill_id) if material_model else []

    user_id = session.get('user_id')
    is_owner = (user_id and str(owner_id) == str(user_id))
    has_pending = False

    if user_id and not is_owner:
        has_pending = request_model.has_pending_request(skill_id, user_id)

    # Annotate materials with user's like state and calculate category counts
    pdf_count = 0
    video_count = 0
    doc_count = 0
    link_count = 0

    for m in materials:
        m['is_liked'] = bool(user_id and str(user_id) in [str(u) for u in m.get('likes', [])])
        m_type = m.get('material_type', 'document')
        if m_type == 'pdf':
            pdf_count += 1
        elif m_type == 'video':
            video_count += 1
        elif m_type == 'document':
            doc_count += 1
        elif m_type == 'link':
            link_count += 1

    return render_template(
        'skill_details.html',
        skill=skill,
        owner=owner,
        reviews=reviews,
        review_stats=review_stats,
        materials=materials,
        pdf_count=pdf_count,
        video_count=video_count,
        doc_count=doc_count,
        link_count=link_count,
        total_materials=len(materials),
```

```
        is_owner=is_owner,
        has_pending=has_pending
    )


# ========================================
# Course Materials & Media Sharing Routes
# ========================================

@app.route('/skill/<skill_id>/materials/upload', methods=['POST'])
@login_required
def upload_material(skill_id):
    """
    Upload and share learning materials (PDFs, Videos, Documents, or Links) within a specific
course/skill.
    """
    skill = skill_model.get_by_id(skill_id)
    if not skill:
        flash('Skill / Course not found.', 'danger')
        return redirect(url_for('skills'))

    user_id = session.get('user_id')
    user_name = session.get('user_name', 'Student Peer')
    owner_id = skill.get('owner_id') or (skill.get('mentor', {}).get('user_id') if
isinstance(skill.get('mentor'), dict) else None)
    is_owner = (str(owner_id) == str(user_id))
    uploader_role = "Course Mentor" if is_owner else "Student Peer"

    title = request.form.get('title', '').strip()
    description = request.form.get('description', '').strip()
    tag = request.form.get('tag', 'Lecture Notes').strip()
    upload_type = request.form.get('upload_type', '').strip()

    file_path = None
    file_name = None
    file_size_formatted = None
    file_ext = None
    external_url = None
    embed_url = None
    detected_type = 'document'

    has_uploaded_file = ('material_file' in request.files and
request.files['material_file'].filename != '')
    has_entered_url = bool(request.form.get('external_url', '').strip())

    if has_uploaded_file or upload_type == 'file':
        if 'material_file' not in request.files:
            flash('No file attached. Please select a PDF or document.', 'warning')
            return redirect(url_for('skill_details', skill_id=skill_id) + '#materials')

        file = request.files['material_file']
        if not file or file.filename == '':
            flash('Please select a PDF or media file to upload.', 'warning')
            return redirect(url_for('skill_details', skill_id=skill_id) + '#materials')

        orig_filename = secure_filename(file.filename)
        if not orig_filename:
            orig_filename = f"resource_{int(datetime.now(timezone.utc).timestamp())}"

        ext = orig_filename.rsplit('.', 1)[-1].lower() if '.' in orig_filename else ''
        if ext not in Config.ALLOWED_EXTENSIONS:
            allowed_list = ", ".join(sorted(list(Config.ALLOWED_EXTENSIONS))).upper()
            flash(f'File type .{ext} is not allowed. Supported formats: {allowed_list}.', 'danger')
            return redirect(url_for('skill_details', skill_id=skill_id) + '#materials')

        if not title:
            base_name = file.filename.rsplit('.', 1)[0].replace('_', ' ').replace('-', '
```

```
').strip().title()
            title = base_name or "Course Learning Resource"

        # Subdirectory per skill
        skill_dir = os.path.join(app.config['UPLOAD_FOLDER'], str(skill['_id']))
        os.makedirs(skill_dir, exist_ok=True)

        timestamp_str = datetime.now(timezone.utc).strftime("%Y%m%d_%H%M%S")
        saved_filename = f"{timestamp_str}_{orig_filename}"
        dest_path = os.path.join(skill_dir, saved_filename)
        file.save(dest_path)

        file_size_bytes = os.path.getsize(dest_path)
        file_size_formatted = format_file_size(file_size_bytes)
        file_path = f"{skill['_id']}/{saved_filename}"
        file_name = file.filename
        file_ext = ext
        detected_type = detect_material_type(filename=orig_filename)

    elif has_entered_url or upload_type == 'link':
        url = request.form.get('external_url', '').strip()
        if not url:
            flash('Please provide a valid URL for the resource or video.', 'warning')
            return redirect(url_for('skill_details', skill_id=skill_id) + '#materials')

        if not (url.startswith('http://') or url.startswith('https://')):
            url = 'https://' + url

        external_url = url
        embed_url, _ = get_youtube_embed_info(url)
        detected_type = detect_material_type(external_url=url)
        file_size_formatted = "Online Media" if detected_type == 'video' else "Web Resource"

        if not title:
            title = "Video Lecture" if detected_type == 'video' else "Web Reference"
    else:
        flash('Please select a PDF file to upload or enter a link.', 'warning')
        return redirect(url_for('skill_details', skill_id=skill_id) + '#materials')

    material_doc, err = material_model.create_material(
        skill_id=skill_id,
        uploader_id=user_id,
        uploader_name=user_name,
        uploader_role=uploader_role,
        title=title,
        description=description,
        material_type=detected_type,
        file_path=file_path,
        file_name=file_name,
        file_size=file_size_formatted,
        file_ext=file_ext,
        external_url=external_url,
        embed_url=embed_url,
        tag=tag
    )

    if err:
        flash(f'Error sharing resource: {err}', 'danger')
    else:
        label = "PDF Document" if detected_type == 'pdf' else ("Video Tutorial" if detected_type ==
'video' else "Learning Material")
        flash(f' Success! "{title}" ({label}) has been added to this course.', 'success')

    return redirect(url_for('skill_details', skill_id=skill_id) + '#materials')


@app.route('/skill/<skill_id>/material/<material_id>/download')
```

```
def download_material(skill_id, material_id):
    """
    Download a shared document/PDF and record download metrics.
    """
    material = material_model.get_by_id(material_id)
    if not material:
        flash('Requested material not found.', 'warning')
        return redirect(url_for('skill_details', skill_id=skill_id))

    material_model.increment_downloads(material_id)

    if material.get('file_path'):
        rel_path = material.get('file_path')
        abs_path = os.path.join(app.config['UPLOAD_FOLDER'], rel_path)
        directory = os.path.dirname(abs_path)
        filename = os.path.basename(abs_path)
        download_name = material.get('file_name') or filename

        if not os.path.exists(abs_path):
            flash('File is currently unavailable on storage server.', 'danger')
            return redirect(url_for('skill_details', skill_id=skill_id) + '#materials')

        return send_from_directory(
            directory,
            filename,
            as_attachment=True,
            download_name=download_name
        )
    elif material.get('external_url'):
        return redirect(material.get('external_url'))
    else:
        flash('No file attached to this material.', 'info')
        return redirect(url_for('skill_details', skill_id=skill_id) + '#materials')


@app.route('/skill/<skill_id>/material/<material_id>/preview')
def preview_material(skill_id, material_id):
    """
    Stream or view media inline in browser modal (for native PDF preview or video player).
    """
    material = material_model.get_by_id(material_id)
    if not material:
        abort(404)

    material_model.increment_views(material_id)

    if material.get('file_path'):
        rel_path = material.get('file_path')
        abs_path = os.path.join(app.config['UPLOAD_FOLDER'], rel_path)
        if not os.path.exists(abs_path):
            abort(404)
        directory = os.path.dirname(abs_path)
        filename = os.path.basename(abs_path)
        return send_from_directory(directory, filename, as_attachment=False)
    elif material.get('external_url'):
        return redirect(material.get('external_url'))
    abort(404)


@app.route('/skill/<skill_id>/material/<material_id>/like', methods=['POST'])
@login_required
def like_material(skill_id, material_id):
    """
    Toggle helpful / upvote reaction on a shared learning material.
    """
    user_id = session.get('user_id')
    success, new_count, liked = material_model.toggle_like(material_id, user_id)
```

```
    if request.headers.get('X-Requested-With') == 'XMLHttpRequest' or request.is_json:
        return jsonify({
            'success': success,
            'likes_count': new_count,
            'liked': liked
        })

    flash('Resource vote updated.', 'success')
    return redirect(url_for('skill_details', skill_id=skill_id) + '#materials')


@app.route('/skill/<skill_id>/material/<material_id>/delete', methods=['POST'])
@login_required
def delete_material(skill_id, material_id):
    """
    Remove a shared course material (allowed for uploader or course mentor).
    """
    skill = skill_model.get_by_id(skill_id)
    if not skill:
        flash('Skill not found.', 'danger')
        return redirect(url_for('skills'))

    user_id = session.get('user_id')
    owner_id = skill.get('owner_id') or (skill.get('mentor', {}).get('user_id') if
isinstance(skill.get('mentor'), dict) else None)
    is_skill_owner = (str(owner_id) == str(user_id))

    success, material_doc, err = material_model.delete_material(
        material_id=material_id,
        user_id=user_id,
        is_skill_owner=is_skill_owner
    )

    if success:
        if material_doc and material_doc.get('file_path'):
            try:
                fpath = os.path.join(app.config['UPLOAD_FOLDER'], material_doc.get('file_path'))
                if os.path.exists(fpath):
                    os.remove(fpath)
            except Exception as e:
                print(f"Warning: could not delete file from disk: {e}")
        flash('Course material has been removed.', 'success')
    else:
        flash(f'Cannot delete material: {err}', 'danger')

    return redirect(url_for('skill_details', skill_id=skill_id) + '#materials')


# ========================================
# Requests Management Routes
# ========================================

@app.route('/requests')
@login_required
def requests_page():
    """
    Manage received requests and outgoing sent requests.
    """
    user_id = session.get('user_id')
    received_requests = request_model.get_received_requests(user_id)
    sent_requests = request_model.get_sent_requests(user_id)

    return render_template(
        'requests.html',
        received_requests=received_requests,
        sent_requests=sent_requests
```

```
    )


@app.route('/request-skill/<skill_id>', methods=['POST'])
@login_required
def request_skill(skill_id):
    """
    Send a learning request to the skill mentor.
    """
    skill = skill_model.get_by_id(skill_id)
    if not skill:
        flash('Skill not found.', 'danger')
        return redirect(url_for('skills'))

    sender_id = session.get('user_id')
    sender_name = session.get('user_name')
    receiver_id = skill.get('owner_id')
    receiver_name = skill.get('owner_name')

    if str(receiver_id) == sender_id:
        flash('You cannot request to learn your own skill.', 'warning')
        return redirect(url_for('skill_details', skill_id=skill_id))

    message = request.form.get('message', '').strip()

    req, err = request_model.create_request(
        skill_id=skill_id,
        skill_title=skill.get('title'),
        sender_id=sender_id,
        sender_name=sender_name,
        receiver_id=receiver_id,
        receiver_name=receiver_name,
        message=message
    )

    if err:
        flash(err, 'warning')
    else:
        flash('Request sent successfully! The mentor has been notified.', 'success')

    return redirect(url_for('skill_details', skill_id=skill_id))


@app.route('/accept-request/<request_id>', methods=['POST'])
@login_required
def accept_request(request_id):
    """
    Accept an incoming learning request.
    """
    user_id = session.get('user_id')
    success, err = request_model.update_status(request_id, 'Accepted', user_id)

    if success:
        flash('Request accepted. You can now coordinate session timings with your peer.', 'success')
    else:
        flash(f'Unable to accept request: {err}', 'danger')

    return redirect(url_for('requests_page'))


@app.route('/reject-request/<request_id>', methods=['POST'])
@login_required
def reject_request(request_id):
    """
    Decline an incoming learning request.
    """
    user_id = session.get('user_id')
```

```
    success, err = request_model.update_status(request_id, 'Rejected', user_id)

    if success:
        flash('Request declined.', 'info')
    else:
        flash(f'Unable to decline request: {err}', 'danger')

    return redirect(url_for('requests_page'))


@app.route('/complete-request/<request_id>', methods=['POST'])
@login_required
def complete_request(request_id):
    """
    Mark an active session as completed.
    """
    user_id = session.get('user_id')
    req = request_model.get_by_id(request_id)
    if not req:
        flash('Request not found.', 'danger')
        return redirect(url_for('requests_page'))

    success, err = request_model.update_status(request_id, 'Completed', user_id)

    if success:
        # Increment skills learned count for student
        user_model.increment_skills_learned(req.get('sender_id'), 1)
        flash('Session marked as completed! You can now leave a rating and review.', 'success')
    else:
        flash(f'Unable to complete request: {err}', 'danger')

    return redirect(url_for('requests_page'))


# ========================================
# Reviews & Ratings Routes
# ========================================

@app.route('/review/<request_id>', methods=['POST'])
@login_required
def submit_review(request_id):
    """
    Submit rating (1-5) and review for a completed learning session.
    """
    user_id = session.get('user_id')
    req = request_model.get_by_id(request_id)

    if not req:
        flash('Request not found.', 'danger')
        return redirect(url_for('requests_page'))

    if str(req.get('sender_id')) != user_id:
        flash('Only the student who requested and learned the skill can review this session.',
'danger')
        return redirect(url_for('requests_page'))

    if req.get('status') != 'Completed':
        flash('Only completed sessions can be reviewed.', 'warning')
        return redirect(url_for('requests_page'))

    rating = request.form.get('rating')
    comment = request.form.get('comment', '').strip()

    review, err = review_model.create_review(
        skill_id=req.get('skill_id'),
        request_id=req.get('_id'),
        reviewer_id=user_id,
```

```
        reviewer_name=session.get('user_name', 'Student'),
        rating=rating,
        comment=comment
    )

    if err:
        flash(err, 'danger')
    else:
        request_model.mark_reviewed(request_id)
        flash('Review submitted successfully! Thank you for supporting peer learning.', 'success')

    return redirect(url_for('skill_details', skill_id=req.get('skill_id')))


@app.route('/reviews')
def reviews_page():
    """
    Public reviews and testimonials gallery.
    """
    if not review_model:
        return render_template('reviews.html', reviews=[])
    all_reviews = list(db.reviews.find().sort("created_at", -1).limit(50))
    return render_template('reviews.html', reviews=all_reviews)


@app.route('/seed-data')
def seed_data_route():
    """
    Optional development helper route to populate initial sample skills and users.
    """
    if db is None:
        return "Database is not connected.", 500
    res = seed_sample_data(db)
    flash(res["message"], res["status"])
    return redirect(url_for('skills'))


# ========================================
# Error Handlers
# ========================================

@app.errorhandler(404)
def page_not_found(e):
    return render_template('404.html'), 404


@app.errorhandler(500)
def server_error(e):
    return render_template('500.html'), 500


if __name__ == '__main__':
    # Check for cli argument to seed data
    if len(sys.argv) > 1 and sys.argv[1] == 'seed':
        if db is not None:
            res = seed_sample_data(db)
            print(res["message"])
        sys.exit(0)

    port = int(os.environ.get("PORT", 5000))
    app.run(host='127.0.0.1', port=port, debug=Config.DEBUG)
```

config.py:

```
import os
from dotenv import load_dotenv
```

```
# Load environment variables from .env file
load_dotenv()

class Config:
    SECRET_KEY = os.getenv("SECRET_KEY", "default-dev-secret-key-change-in-production")
    MONGO_URI = os.getenv("MONGO_URI", "mongodb://localhost:27017/")
    DATABASE_NAME = os.getenv("DATABASE_NAME", "student_skill_sharing")
    DEBUG = os.getenv("FLASK_DEBUG", "True").lower() in ("true", "1", "t")

    # Categories supported
    CATEGORIES = [
        "Programming",
        "Web Development",
        "Mobile Development",
        "Database",
        "Data Science",
        "Artificial Intelligence",
        "Design",
        "Communication",
        "Marketing",
        "Music",
        "Sports",
        "Photography",
        "Other"
    ]

    # Skill levels
    SKILL_LEVELS = [
        "Beginner",
        "Intermediate",
        "Advanced"
    ]

    # Department list for college student profile
    DEPARTMENTS = [
        "Computer Science & Engineering",
        "Information Technology",
        "Electronics & Communication",
        "Electrical & Electronics",
        "Mechanical Engineering",
        "Civil Engineering",
        "Data Science & AI",
        "Business Administration",
        "Design & Media",
        "Other"
    ]

    YEARS = ["1st Year", "2nd Year", "3rd Year", "4th Year", "Postgraduate"]

    # Course material upload settings
    UPLOAD_FOLDER = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'static', 'uploads',
'materials')
    MAX_CONTENT_LENGTH = 50 * 1024 * 1024  # 50 MB max file upload
    ALLOWED_EXTENSIONS = {'pdf', 'mp4', 'webm', 'mov', 'mkv', 'doc', 'docx', 'ppt', 'pptx', 'txt',
'zip'}
    RESOURCE_TAGS = [
        "Lecture Notes",
        "Video Tutorial",
        "Cheat Sheet",
        "Lab Exercise",
        "Slide Deck",
        "Reference Code",
        "Practice Problems",
        "Study Guide",
        "Other"
    ]
```

requirements.txt:

```
Flask>=3.0.0
pymongo[srv]>=4.6.0
python-dotenv>=1.0.0
Werkzeug>=3.0.0
```

.env.example:

```
MONGO_URI=mongodb://localhost:27017/
DATABASE_NAME=student_skill_sharing
SECRET_KEY=change_this_secret_key
FLASK_ENV=development
FLASK_DEBUG=1
```

models/__init__.py:

```
from .user_model import UserModel
from .skill_model import SkillModel
from .request_model import RequestModel
from .review_model import ReviewModel
from .material_model import MaterialModel

__all__ = ["UserModel", "SkillModel", "RequestModel", "ReviewModel", "MaterialModel"]
```

models/user_model.py:

```
from datetime import datetime, timezone
from werkzeug.security import generate_password_hash, check_password_hash
from utils.helpers import parse_object_id

class UserModel:
    def __init__(self, db):
        self.collection = db.users

    def create_user(self, name, email, password, department, year, bio="", profile_image=""):
        """
        Create a new user document with hashed password.
        """
        email_clean = email.strip().lower()
        if self.get_by_email(email_clean):
            return None, "An account with this email already exists."

        if not profile_image or not profile_image.strip():
            # Clean initial-based avatar fallback or modern illustrated avatar
            profile_image = f"https://api.dicebear.com/7.x/initials/svg?seed={name.replace(' ',
'+')}&backgroundColor=6366f1,8b5cf6,4f46e5&textColor=ffffff"

        user_doc = {
            "name": name.strip(),
            "email": email_clean,
            "password_hash": generate_password_hash(password),
            "department": department.strip(),
            "year": year.strip(),
            "bio": bio.strip() if bio else "Eager to share skills and learn from fellow students!",
            "profile_image": profile_image.strip(),
            "skills_offered": 0,
            "skills_learned": 0,
            "created_at": datetime.now(timezone.utc)
        }

        result = self.collection.insert_one(user_doc)
        user_doc["_id"] = result.inserted_id
        return user_doc, None

    def get_by_email(self, email):
        """
```

```
        Find user by lowercase email.
        """
        if not email:
            return None
        return self.collection.find_one({"email": email.strip().lower()})

    def get_by_id(self, user_id):
        """
        Find user by ObjectId.
        """
        oid = parse_object_id(user_id)
        if not oid:
            return None
        return self.collection.find_one({"_id": oid})

    def verify_password(self, user_doc, password):
        """
        Verify plain password against stored password hash.
        """
        if not user_doc or "password_hash" not in user_doc:
            return False
        return check_password_hash(user_doc["password_hash"], password)

    def update_profile(self, user_id, name, department, year, bio, profile_image=None):
        """
        Update user profile information.
        """
        oid = parse_object_id(user_id)
        if not oid:
            return False, "Invalid user ID"

        update_fields = {
            "name": name.strip(),
            "department": department.strip(),
            "year": year.strip(),
            "bio": bio.strip(),
            "updated_at": datetime.now(timezone.utc)
        }

        if profile_image and profile_image.strip():
            update_fields["profile_image"] = profile_image.strip()

        result = self.collection.update_one({"_id": oid}, {"$set": update_fields})
        return result.modified_count >= 0, None

    def increment_skills_offered(self, user_id, delta=1):
        """
        Update skills offered count for user.
        """
        oid = parse_object_id(user_id)
        if oid:
            self.collection.update_one({"_id": oid}, {"$inc": {"skills_offered": delta}})

    def increment_skills_learned(self, user_id, delta=1):
        """
        Update skills learned count for user.
        """
        oid = parse_object_id(user_id)
        if oid:
            self.collection.update_one({"_id": oid}, {"$inc": {"skills_learned": delta}})
```

models/skill_model.py:

```
from datetime import datetime, timezone
import re
from utils.helpers import parse_object_id
```

```
class SkillModel:
    def __init__(self, db):
        self.collection = db.skills

    def create_skill(self, title, description, category, level, owner_id, owner_name):
        """
        Create a new skill document.
        """
        oid = parse_object_id(owner_id)
        if not oid:
            return None, "Invalid owner ID"

        skill_doc = {
            "title": title.strip(),
            "description": description.strip(),
            "category": category.strip(),
            "level": level.strip(),
            "owner_id": oid,
            "owner_name": owner_name.strip(),
            "created_at": datetime.now(timezone.utc)
        }

        result = self.collection.insert_one(skill_doc)
        skill_doc["_id"] = result.inserted_id
        return skill_doc, None

    def get_by_id(self, skill_id):
        """
        Find skill by ObjectId.
        """
        oid = parse_object_id(skill_id)
        if not oid:
            return None
        return self.collection.find_one({"_id": oid})

    def get_all_skills(self, search=None, category=None, level=None, limit=100):
        """
        Search and filter skills with case-insensitive matching.
        """
        query = {}

        if category and category.strip() and category != "All":
            query["category"] = category.strip()

        if level and level.strip() and level != "All":
            query["level"] = level.strip()

        if search and search.strip():
            search_regex = re.compile(re.escape(search.strip()), re.IGNORECASE)
            query["$or"] = [
                {"title": {"$regex": search_regex}},
                {"description": {"$regex": search_regex}},
                {"owner_name": {"$regex": search_regex}},
                {"category": {"$regex": search_regex}}
            ]

        cursor = self.collection.find(query).sort("created_at", -1)
        if limit:
            cursor = cursor.limit(limit)
        return list(cursor)

    def get_by_owner_id(self, owner_id):
        """
        Get all skills posted by a specific user.
        """
        oid = parse_object_id(owner_id)
        if not oid:
```

```
            return []
        return list(self.collection.find({"owner_id": oid}).sort("created_at", -1))

    def count_by_owner(self, owner_id):
        """
        Count skills posted by a specific user.
        """
        oid = parse_object_id(owner_id)
        if not oid:
            return 0
        return self.collection.count_documents({"owner_id": oid})

    def get_recent_skills(self, limit=6):
        """
        Get the most recently posted skills.
        """
        return list(self.collection.find().sort("created_at", -1).limit(limit))

    def update_skill(self, skill_id, owner_id, title, description, category, level):
        """
        Update skill document ensuring owner authorization.
        """
        s_oid = parse_object_id(skill_id)
        o_oid = parse_object_id(owner_id)

        if not s_oid or not o_oid:
            return False, "Invalid IDs"

        # Check ownership
        skill = self.collection.find_one({"_id": s_oid})
        if not skill:
            return False, "Skill not found"
        if skill.get("owner_id") != o_oid:
            return False, "Unauthorized: You can only edit your own skills"

        update_fields = {
            "title": title.strip(),
            "description": description.strip(),
            "category": category.strip(),
            "level": level.strip(),
            "updated_at": datetime.now(timezone.utc)
        }

        self.collection.update_one({"_id": s_oid}, {"$set": update_fields})
        return True, None

    def delete_skill(self, skill_id, owner_id):
        """
        Delete a skill document with ownership check.
        """
        s_oid = parse_object_id(skill_id)
        o_oid = parse_object_id(owner_id)

        if not s_oid or not o_oid:
            return False, "Invalid IDs"

        skill = self.collection.find_one({"_id": s_oid})
        if not skill:
            return False, "Skill not found"
        if skill.get("owner_id") != o_oid:
            return False, "Unauthorized: You can only delete your own skills"

        self.collection.delete_one({"_id": s_oid})
        return True, None
```

models/request_model.py:

```
from datetime import datetime, timezone
from utils.helpers import parse_object_id

class RequestModel:
    def __init__(self, db):
        self.collection = db.requests

    def create_request(self, skill_id, skill_title, sender_id, sender_name, receiver_id,
receiver_name, message):
        """
        Create a new skill learning request.
        """
        s_oid = parse_object_id(skill_id)
        sender_oid = parse_object_id(sender_id)
        receiver_oid = parse_object_id(receiver_id)

        if not s_oid or not sender_oid or not receiver_oid:
            return None, "Invalid ID format"

        if sender_oid == receiver_oid:
            return None, "You cannot request to learn your own skill."

        # Check for existing pending request
        if self.has_pending_request(s_oid, sender_oid):
            return None, "You already have a pending request for this skill."

        req_doc = {
            "skill_id": s_oid,
            "skill_title": skill_title.strip(),
            "sender_id": sender_oid,
            "sender_name": sender_name.strip(),
            "receiver_id": receiver_oid,
            "receiver_name": receiver_name.strip(),
            "message": message.strip() if message else "Hi! I would like to connect and learn this
skill.",
            "status": "Pending",
            "reviewed": False,
            "created_at": datetime.now(timezone.utc)
        }

        result = self.collection.insert_one(req_doc)
        req_doc["_id"] = result.inserted_id
        return req_doc, None

    def get_by_id(self, request_id):
        """
        Get request document by ObjectId.
        """
        oid = parse_object_id(request_id)
        if not oid:
            return None
        return self.collection.find_one({"_id": oid})

    def has_pending_request(self, skill_id, sender_id):
        """
        Check if user has an active pending request for this skill.
        """
        s_oid = parse_object_id(skill_id)
        u_oid = parse_object_id(sender_id)
        if not s_oid or not u_oid:
            return False

        return self.collection.find_one({
            "skill_id": s_oid,
            "sender_id": u_oid,
```

```
            "status": "Pending"
        }) is not None

    def get_sent_requests(self, user_id):
        """
        Get all requests sent by this user.
        """
        oid = parse_object_id(user_id)
        if not oid:
            return []
        return list(self.collection.find({"sender_id": oid}).sort("created_at", -1))

    def get_received_requests(self, user_id):
        """
        Get all requests received by this user.
        """
        oid = parse_object_id(user_id)
        if not oid:
            return []
        return list(self.collection.find({"receiver_id": oid}).sort("created_at", -1))

    def update_status(self, request_id, new_status, user_id):
        """
        Update request status with strict authorization checks.
        - Accept/Reject: Must be receiver (skill owner)
        - Complete: Can be receiver or sender when in Accepted status
        """
        r_oid = parse_object_id(request_id)
        u_oid = parse_object_id(user_id)

        if not r_oid or not u_oid:
            return False, "Invalid IDs"

        req = self.collection.find_one({"_id": r_oid})
        if not req:
            return False, "Request not found"

        if new_status in ["Accepted", "Rejected"]:
            if req.get("receiver_id") != u_oid:
                return False, "Unauthorized: Only the skill owner can accept or reject this request."
            if req.get("status") != "Pending":
                return False, f"Request cannot be modified as it is currently '{req.get('status')}'."

        elif new_status == "Completed":
            if req.get("receiver_id") != u_oid and req.get("sender_id") != u_oid:
                return False, "Unauthorized: You are not a participant of this request."
            if req.get("status") != "Accepted":
                return False, "Only accepted sessions can be marked as completed."

        update_fields = {"status": new_status, "updated_at": datetime.now(timezone.utc)}
        if new_status == "Completed":
            update_fields["completed_at"] = datetime.now(timezone.utc)

        self.collection.update_one({"_id": r_oid}, {"$set": update_fields})
        return True, None

    def mark_reviewed(self, request_id):
        """
        Mark a completed request as reviewed.
        """
        oid = parse_object_id(request_id)
        if oid:
            self.collection.update_one({"_id": oid}, {"$set": {"reviewed": True}})

    def get_user_stats(self, user_id):
        """
        Retrieve count statistics for user dashboard.
```

```
        """
        oid = parse_object_id(user_id)
        if not oid:
            return {"total_learning_requests": 0, "pending_requests": 0, "completed_sessions": 0}

        total_sent = self.collection.count_documents({"sender_id": oid})
        pending_received = self.collection.count_documents({"receiver_id": oid, "status": "Pending"})
        pending_sent = self.collection.count_documents({"sender_id": oid, "status": "Pending"})

        # Completed sessions as learner or teacher
        completed_sessions = self.collection.count_documents({
            "$or": [{"sender_id": oid}, {"receiver_id": oid}],
            "status": "Completed"
        })

        return {
            "total_learning_requests": total_sent,
            "pending_received": pending_received,
            "pending_sent": pending_sent,
            "total_pending": pending_received + pending_sent,
            "completed_sessions": completed_sessions
        }
```

models/review_model.py:

```
from datetime import datetime, timezone
from utils.helpers import parse_object_id

class ReviewModel:
    def __init__(self, db):
        self.collection = db.reviews

    def create_review(self, skill_id, request_id, reviewer_id, reviewer_name, rating, comment):
        """
        Create a new review for a completed skill sharing request.
        """
        s_oid = parse_object_id(skill_id)
        r_oid = parse_object_id(request_id)
        u_oid = parse_object_id(reviewer_id)

        if not s_oid or not r_oid or not u_oid:
            return None, "Invalid IDs provided"

        # Prevent duplicate review on the same completed request
        if self.has_reviewed_request(r_oid):
            return None, "You have already reviewed this learning session."

        try:
            rating_int = int(rating)
            if rating_int < 1 or rating_int > 5:
                return None, "Rating must be between 1 and 5 stars."
        except (ValueError, TypeError):
            return None, "Invalid rating value."

        review_doc = {
            "skill_id": s_oid,
            "request_id": r_oid,
            "reviewer_id": u_oid,
            "reviewer_name": reviewer_name.strip(),
            "rating": rating_int,
            "comment": comment.strip() if comment else "",
            "created_at": datetime.now(timezone.utc)
        }

        result = self.collection.insert_one(review_doc)
        review_doc["_id"] = result.inserted_id
        return review_doc, None
```

```
    def get_by_skill_id(self, skill_id):
        """
        Fetch all reviews for a specific skill.
        """
        oid = parse_object_id(skill_id)
        if not oid:
            return []
        return list(self.collection.find({"skill_id": oid}).sort("created_at", -1))

    def get_by_request_id(self, request_id):
        """
        Fetch review for a specific request.
        """
        oid = parse_object_id(request_id)
        if not oid:
            return None
        return self.collection.find_one({"request_id": oid})

    def has_reviewed_request(self, request_id):
        """
        Check if a request was already reviewed.
        """
        oid = parse_object_id(request_id)
        if not oid:
            return False
        return self.collection.find_one({"request_id": oid}) is not None

    def get_skill_stats(self, skill_id):
        """
        Calculate average rating and total reviews for a skill.
        """
        oid = parse_object_id(skill_id)
        if not oid:
            return {"average_rating": 0.0, "total_reviews": 0}

        reviews = list(self.collection.find({"skill_id": oid}))
        if not reviews:
            return {"average_rating": 0.0, "total_reviews": 0}

        total = len(reviews)
        avg = sum(r["rating"] for r in reviews) / total
        return {
            "average_rating": round(avg, 1),
            "total_reviews": total
        }
```

models/material_model.py:

```
from datetime import datetime, timezone
from utils.helpers import parse_object_id

class MaterialModel:
    def __init__(self, db):
        self.collection = db.materials

    def create_material(self, skill_id, uploader_id, uploader_name, uploader_role,
                        title, description, material_type, file_path=None, file_name=None,
                        file_size=None, file_ext=None, external_url=None, embed_url=None, tag="Lecture
Notes"):
        """
        Create a new course learning material document (PDF, Video, Document, Link).
        """
        s_oid = parse_object_id(skill_id)
        u_oid = parse_object_id(uploader_id)

        if not s_oid or not u_oid:
```

```
            return None, "Invalid Skill or User ID"

        material_doc = {
            "skill_id": s_oid,
            "uploader_id": u_oid,
            "uploader_name": uploader_name.strip(),
            "uploader_role": uploader_role,  # 'mentor' or 'student'
            "title": title.strip(),
            "description": (description or "").strip(),
            "material_type": material_type.lower().strip(),  # 'pdf', 'video', 'document', 'link'
            "file_path": file_path,
            "file_name": file_name,
            "file_size": file_size,
            "file_ext": (file_ext or "").lower().strip(),
            "external_url": (external_url or "").strip() or None,
            "embed_url": (embed_url or "").strip() or None,
            "tag": tag.strip() if tag else "General Resource",
            "likes": [],
            "downloads_count": 0,
            "views_count": 0,
            "created_at": datetime.now(timezone.utc)
        }

        result = self.collection.insert_one(material_doc)
        material_doc["_id"] = result.inserted_id
        return material_doc, None

    def get_by_id(self, material_id):
        """
        Retrieve a single material by ObjectId.
        """
        oid = parse_object_id(material_id)
        if not oid:
            return None
        return self.collection.find_one({"_id": oid})

    def get_by_skill_id(self, skill_id, material_type=None):
        """
        Retrieve all materials shared within a specific skill/course.
        """
        oid = parse_object_id(skill_id)
        if not oid:
            return []

        query = {"skill_id": oid}
        if material_type and material_type.lower() != 'all':
            query["material_type"] = material_type.lower()

        return list(self.collection.find(query).sort("created_at", -1))

    def count_by_skill(self, skill_id):
        """
        Count total materials uploaded to a skill.
        """
        oid = parse_object_id(skill_id)
        if not oid:
            return 0
        return self.collection.count_documents({"skill_id": oid})

    def get_by_uploader(self, uploader_id, limit=20):
        """
        Retrieve all materials uploaded by a specific user across courses.
        """
        oid = parse_object_id(uploader_id)
        if not oid:
            return []
        cursor = self.collection.find({"uploader_id": oid}).sort("created_at", -1)
```

```
        if limit:
            cursor = cursor.limit(limit)
        return list(cursor)

    def count_by_uploader(self, uploader_id):
        """
        Count total materials shared by a student.
        """
        oid = parse_object_id(uploader_id)
        if not oid:
            return 0
        return self.collection.count_documents({"uploader_id": oid})

    def toggle_like(self, material_id, user_id):
        """
        Toggle helpful/upvote reaction for a material by a student.
        """
        m_oid = parse_object_id(material_id)
        u_str = str(user_id)

        if not m_oid or not u_str:
            return False, 0, "Invalid ID"

        material = self.collection.find_one({"_id": m_oid})
        if not material:
            return False, 0, "Material not found"

        likes = material.get("likes", [])
        if u_str in likes:
            self.collection.update_one({"_id": m_oid}, {"$pull": {"likes": u_str}})
            new_count = len(likes) - 1
            return True, new_count, False  # unliked
        else:
            self.collection.update_one({"_id": m_oid}, {"$addToSet": {"likes": u_str}})
            new_count = len(likes) + 1
            return True, new_count, True  # liked

    def increment_downloads(self, material_id):
        """
        Increment the download counter when a student downloads a document.
        """
        m_oid = parse_object_id(material_id)
        if m_oid:
            self.collection.update_one({"_id": m_oid}, {"$inc": {"downloads_count": 1}})

    def increment_views(self, material_id):
        """
        Increment the view/preview counter when a student previews or watches media.
        """
        m_oid = parse_object_id(material_id)
        if m_oid:
            self.collection.update_one({"_id": m_oid}, {"$inc": {"views_count": 1}})

    def delete_material(self, material_id, user_id, is_skill_owner=False):
        """
        Delete a material if the requester is the uploader or the skill owner (mentor).
        Returns (success, deleted_material_doc, error_message)
        """
        m_oid = parse_object_id(material_id)
        u_oid = parse_object_id(user_id)

        if not m_oid or not u_oid:
            return False, None, "Invalid ID"

        material = self.collection.find_one({"_id": m_oid})
        if not material:
            return False, None, "Material not found"
```

```
        # Permission check: must be either the original uploader or the course owner
        if str(material.get("uploader_id")) != str(u_oid) and not is_skill_owner:
            return False, None, "Unauthorized: Only the uploader or course mentor can delete this
material"

        self.collection.delete_one({"_id": m_oid})
        return True, material, None
```

utils/helpers.py (utility functions):

```
from functools import wraps
from flask import session, redirect, url_for, flash, request
from bson import ObjectId
from datetime import datetime, timezone
import pymongo
from werkzeug.security import generate_password_hash

def login_required(f):
    """
    Decorator to ensure user is logged in before accessing a route.
    """
    @wraps(f)
    def decorated_function(*args, **kwargs):
        if 'user_id' not in session:
            flash('Please log in to access this page.', 'warning')
            return redirect(url_for('login', next=request.url))
        return f(*args, **kwargs)
    return decorated_function

def parse_object_id(id_str):
    """
    Safely parse a string into a BSON ObjectId.
    Returns ObjectId or None if invalid.
    """
    try:
        if isinstance(id_str, ObjectId):
            return id_str
        return ObjectId(str(id_str))
    except Exception:
        return None

def format_datetime(value, format="%b %d, %Y"):
    """
    Format a datetime object for Jinja2 templates.
    """
    if not value:
        return ""
    if isinstance(value, str):
        try:
            value = datetime.fromisoformat(value)
        except Exception:
            return value
    if isinstance(value, datetime):
        return value.strftime(format)
    return str(value)

def time_ago(value):
    """
    Display a friendly relative time (e.g., '2 hours ago', 'Just now').
    """
    if not value:
        return ""
    if isinstance(value, str):
        try:
            value = datetime.fromisoformat(value)
        except Exception:
```

```
            return value
        if not isinstance(value, datetime):
            return str(value)

        now = datetime.now(timezone.utc)
        if value.tzinfo is None:
            value = value.replace(tzinfo=timezone.utc)

        diff = now - value

        seconds = diff.total_seconds()
        if seconds < 0:
            seconds = 0
        if seconds < 60:
            return "Just now"
        elif seconds < 3600:
            minutes = int(seconds // 60)
            return f"{minutes} minute{'s' if minutes > 1 else ''} ago"
        elif seconds < 86400:
            hours = int(seconds // 3600)
            return f"{hours} hour{'s' if hours > 1 else ''} ago"
        elif seconds < 604800:
            days = int(seconds // 86400)
            return f"{days} day{'s' if days > 1 else ''} ago"
        else:
            return value.strftime("%b %d, %Y")

    def format_file_size(bytes_size):
        """
        Format bytes into human-readable size string (KB, MB, GB).
        """
        if not bytes_size or bytes_size <= 0:
            return "0 B"
        units = ["B", "KB", "MB", "GB"]
        i = 0
        size = float(bytes_size)
        while size >= 1024.0 and i < len(units) - 1:
            size /= 1024.0
            i += 1
        return f"{size:.1f} {units[i]}"

    def get_youtube_embed_info(url):
        """
        Extract YouTube video ID, embed URL, and thumbnail URL from various YouTube link formats.
        """
        import re
        if not url:
            return None, None
        url = url.strip()
        patterns = [
            r'(?:https?:\/\/)?(?:www\.)?youtube\.com\/watch\?v=([a-zA-Z0-9_-]{11})',
            r'(?:https?:\/\/)?(?:www\.)?youtu\.be\/([a-zA-Z0-9_-]{11})',
            r'(?:https?:\/\/)?(?:www\.)?youtube\.com\/embed\/([a-zA-Z0-9_-]{11})',
            r'(?:https?:\/\/)?(?:www\.)?youtube\.com\/shorts\/([a-zA-Z0-9_-]{11})'
        ]
        for pattern in patterns:
            match = re.search(pattern, url)
            if match:
                video_id = match.group(1)
                embed_url = f"https://www.youtube-nocookie.com/embed/{video_id}"
                thumbnail_url = f"https://img.youtube.com/vi/{video_id}/hqdefault.jpg"
                return embed_url, thumbnail_url
        return None, None

    def detect_material_type(filename=None, external_url=None):
        """
        Detect resource type category: 'pdf', 'video', 'document', or 'link'.
```

```
    """
    if filename:
        ext = filename.rsplit('.', 1)[-1].lower() if '.' in filename else ''
        if ext == 'pdf':
            return 'pdf'
        elif ext in {'mp4', 'webm', 'mov', 'mkv', 'avi'}:
            return 'video'
        elif ext in {'doc', 'docx', 'ppt', 'pptx', 'txt', 'zip'}:
            return 'document'
    if external_url:
        u_lower = external_url.lower()
        if any(k in u_lower for k in ['youtube.com', 'youtu.be', 'vimeo.com', '.mp4', '.webm']):
            return 'video'
        return 'link'
    return 'document'

def init_db_indexes(db):
    """
    Create MongoDB indexes for performance and uniqueness.
    """
    try:
        # User indexes
        db.users.create_index([("email", pymongo.ASCENDING)], unique=True)

        # Skill indexes
        db.skills.create_index([("title", pymongo.TEXT), ("description", pymongo.TEXT)])
        db.skills.create_index([("category", pymongo.ASCENDING)])
        db.skills.create_index([("level", pymongo.ASCENDING)])
        db.skills.create_index([("owner_id", pymongo.ASCENDING)])

        # Request indexes
        db.requests.create_index([("sender_id", pymongo.ASCENDING)])
        db.requests.create_index([("receiver_id", pymongo.ASCENDING)])
        db.requests.create_index([("skill_id", pymongo.ASCENDING)])
        db.requests.create_index([("status", pymongo.ASCENDING)])

        # Review indexes
        db.reviews.create_index([("skill_id", pymongo.ASCENDING)])
        db.reviews.create_index([("request_id", pymongo.ASCENDING)], unique=True)

        # Material indexes
        db.materials.create_index([("skill_id", pymongo.ASCENDING)])
        db.materials.create_index([("uploader_id", pymongo.ASCENDING)])
        db.materials.create_index([("material_type", pymongo.ASCENDING)])
    except Exception as e:
        print(f"Warning: Could not create some indexes: {e}")
```

templates/base.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>{% block title %}Student Skill Sharing Platform{% endblock %} | SkillShare Hub</title>
  <meta name="description" content="A collaborative college laboratory platform for students to
exchange skills, mentor peers, and grow together.">
  <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
</head>
<body>

  <!-- Navigation Bar -->
  <nav class="navbar">
    <div class="container navbar-inner">
      <a href="{{ url_for('index') }}" class="brand-logo" id="nav-brand-logo">
        <div class="brand-icon"></div>
        <span>SkillExchange</span>
```

```
      </a>

      <ul class="nav-links">
        <li><a href="{{ url_for('index') }}" class="nav-link {% if request.endpoint == 'index'
%}active{% endif %}" id="nav-link-home">Home</a></li>
        <li><a href="{{ url_for('skills') }}" class="nav-link {% if request.endpoint in ['skills',
'skill_details'] %}active{% endif %}" id="nav-link-skills">Explore Skills</a></li>

        {% if session.get('user_id') %}
          <li><a href="{{ url_for('dashboard') }}" class="nav-link {% if request.endpoint ==
'dashboard' %}active{% endif %}" id="nav-link-dashboard">Dashboard</a></li>
          <li><a href="{{ url_for('requests_page') }}" class="nav-link {% if request.endpoint ==
'requests_page' %}active{% endif %}" id="nav-link-requests">Requests</a></li>
          <li><a href="{{ url_for('add_skill') }}" class="btn btn-primary btn-sm" id="nav-btn-add-
skill">+ Offer Skill</a></li>
          <li>
            <a href="{{ url_for('profile') }}" class="nav-user-badge" id="nav-link-profile">
              <img src="{{ session.get('user_image', 'https://api.dicebear.com/7.x/initials/svg?seed='
+ (session.get('user_name', 'User')|urlencode)) }}" alt="Profile" class="nav-avatar-mini">
              <span>{{ session.get('user_name', 'Profile') }}</span>
            </a>
          </li>
          <li><a href="{{ url_for('logout') }}" class="btn btn-secondary btn-sm" id="nav-btn-
logout">Logout</a></li>
        {% else %}
          <li><a href="{{ url_for('login') }}" class="btn btn-secondary btn-sm" id="nav-btn-
login">Login</a></li>
          <li><a href="{{ url_for('register') }}" class="btn btn-primary btn-sm" id="nav-btn-
register">Register</a></li>
        {% endif %}
      </ul>
    </div>
  </nav>

  <!-- Flash Messages Notification Area -->
  <div class="container" style="margin-top: 1.5rem;">
    {% with messages = get_flashed_messages(with_categories=true) %}
      {% if messages %}
        <div class="flash-container">
          {% for category, message in messages %}
            <div class="alert alert-{{ category if category != 'error' else 'danger' }}" role="alert">
              <span>{{ message }}</span>
              <button type="button" class="alert-close"
onclick="this.parentElement.style.display='none';">&times;</button>
            </div>
          {% endfor %}
        </div>
      {% endif %}
    {% endwith %}
  </div>

  <!-- Main Content Body -->
  <main class="main-content">
    {% block content %}
    {% endblock %}
  </main>

  <!-- Footer -->
  <footer class="footer">
    <div class="container footer-inner">
      <div>
        <p><strong>Student Skill Sharing Platform</strong> &bull; College Laboratory Project</p>
        <p style="font-size: 0.8rem; margin-top: 0.2rem; color: var(--text-muted);">Built with Python
Flask, Jinja2 & MongoDB</p>
      </div>
      <ul class="footer-links">
        <li><a href="{{ url_for('index') }}" class="footer-link">Home</a></li>
```

```
        <li><a href="{{ url_for('skills') }}" class="footer-link">Browse Skills</a></li>
        {% if not session.get('user_id') %}
          <li><a href="{{ url_for('login') }}" class="footer-link">Sign In</a></li>
          <li><a href="{{ url_for('register') }}" class="footer-link">Create Account</a></li>
        {% else %}
          <li><a href="{{ url_for('dashboard') }}" class="footer-link">Dashboard</a></li>
          <li><a href="{{ url_for('profile') }}" class="footer-link">My Profile</a></li>
        {% endif %}
      </ul>
    </div>
  </footer>

</body>
</html>
```

templates/login.html:

```
{% extends "base.html" %}

{% block title %}Sign In - Student Skill Sharing{% endblock %}

{% block content %}
<div class="container container-sm">
  <div class="card" style="margin-top: 1.5rem;">
    <div style="text-align: center; margin-bottom: 2rem;">
      <div class="brand-icon" style="margin: 0 auto 1rem auto; width: 48px; height: 48px; font-size:
1.5rem;"></div>
      <h2 style="font-size: 1.75rem; font-weight: 800; color: var(--text-main);">Welcome Back</h2>
      <p style="color: var(--text-muted); font-size: 0.9rem; margin-top: 0.25rem;">Sign in to your
SkillExchange account</p>
    </div>

    <form method="POST" action="{{ url_for('login') }}" id="login-form">
      <div class="form-group">
        <label for="email" class="form-label">College Email Address</label>
        <input type="email" id="email" name="email" class="form-control"
placeholder="student@college.edu" required autofocus>
      </div>

      <div class="form-group">
        <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom:
0.45rem;">
          <label for="password" class="form-label" style="margin-bottom: 0;">Password</label>
        </div>
        <input type="password" id="password" name="password" class="form-control" placeholder="Enter
your password" required>
      </div>

      <button type="submit" class="btn btn-primary btn-block btn-lg" id="login-submit-btn"
style="margin-top: 1rem;">
        Sign In &rarr;
      </button>
    </form>

    <div style="text-align: center; margin-top: 1.5rem; padding-top: 1.25rem; border-top: 1px solid
var(--border-color); font-size: 0.9rem; color: var(--text-muted);">
      Don't have an account yet? <a href="{{ url_for('register') }}" style="color: var(--primary);
font-weight: 600; text-decoration: none;">Register now</a>
    </div>
  </div>
</div>
{% endblock %}
```

2.3 DATABASE DESIGN

The application stores its data in a MongoDB database named student_skill_sharing, which contains five collections.

Collection	Main fields	Indexes
users	name, email, password_hash, department, year, bio, profile_image, skills_offered, skills_learned, created_at	email (unique)
skills	title, description, category, level, owner_id, owner_name, created_at	title/description (text), category, level, owner_id
requests	skill_id, skill_title, sender_id, receiver_id, message, status, reviewed, created_at, completed_at	sender_id, receiver_id, skill_id, status
reviews	skill_id, request_id, reviewer_id, reviewer_name, rating (1-5), comment, created_at	skill_id, request_id (unique)
materials	skill_id, uploader_id, uploader_role, title, material_type, file_path, external_url, tag, likes, downloads_count, views_count	skill_id, uploader_id, material_type

2.4 APPLICATION ROUTES

Route	Method	Purpose
/	GET	Home page with recent skills
/register, /login, /logout	GET, POST	Account creation and session handling
/dashboard, /profile	GET, POST	Student statistics, profile view and edit
/skills	GET	Search and filter all skills
/add-skill, /edit-skill/<id>, /delete-skill/<id>	GET, POST	Manage own skill listings
/skill/<id>	GET	Skill details, mentor, reviews and materials
/skill/<id>/materials/upload	POST	Share a PDF, document or link
/skill/<id>/material/<mid>/download, /preview, /like, /delete	GET, POST	Download, preview, upvote or remove material
/requests, /request-skill/<id>	GET, POST	View and send learning requests
/accept-request, /reject-request, /complete-request	POST	Update request status
/review/<request_id>, /reviews	POST, GET	Submit and display reviews

CHAPTER 3
IMPLEMENTATION RESULTS

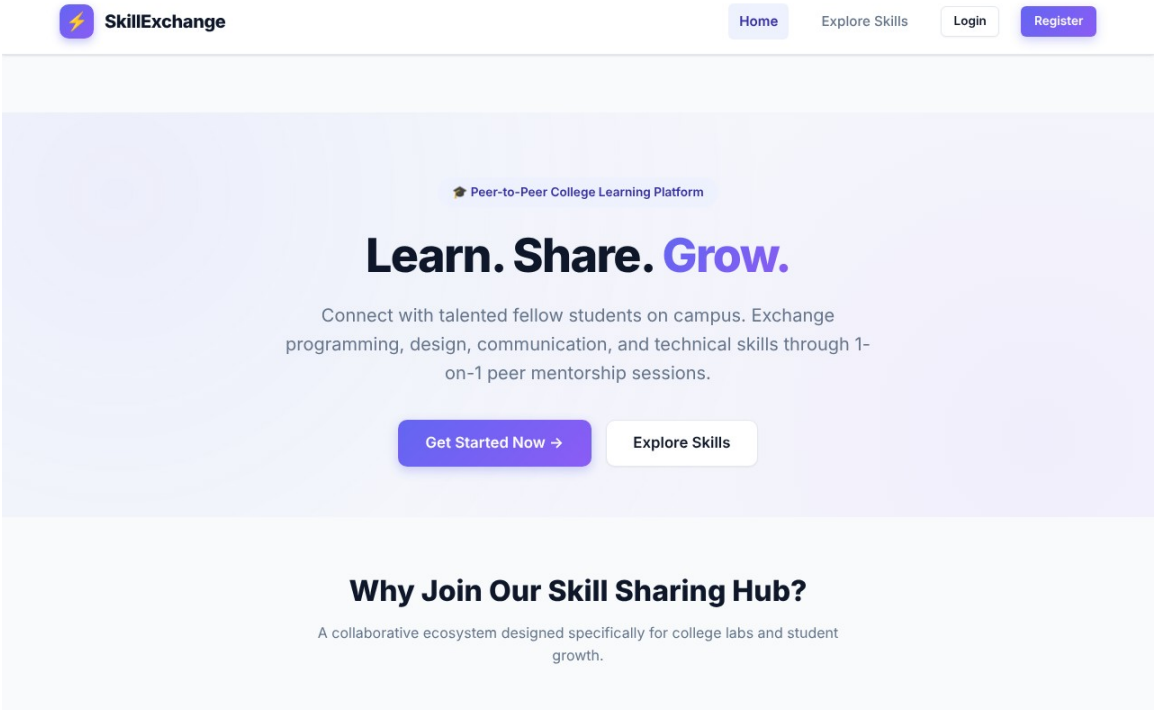


Figure 3.1: Student Skill Exchange Platform Home Page

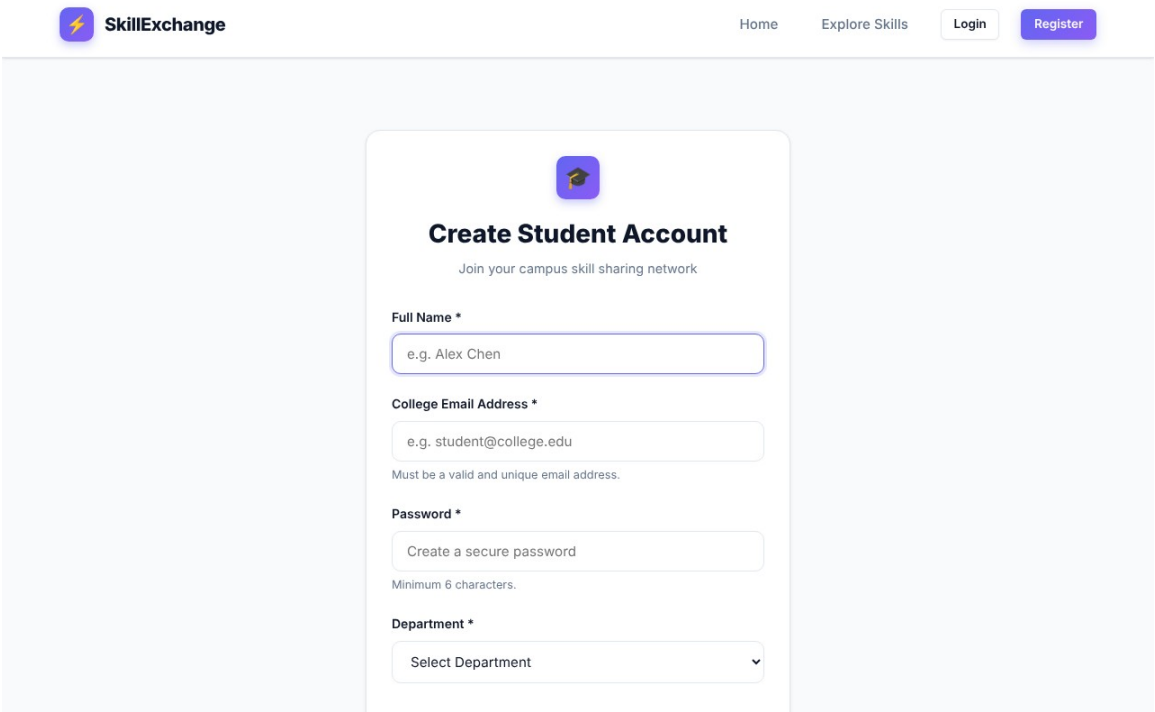


Figure 3.2: Student Registration Interface

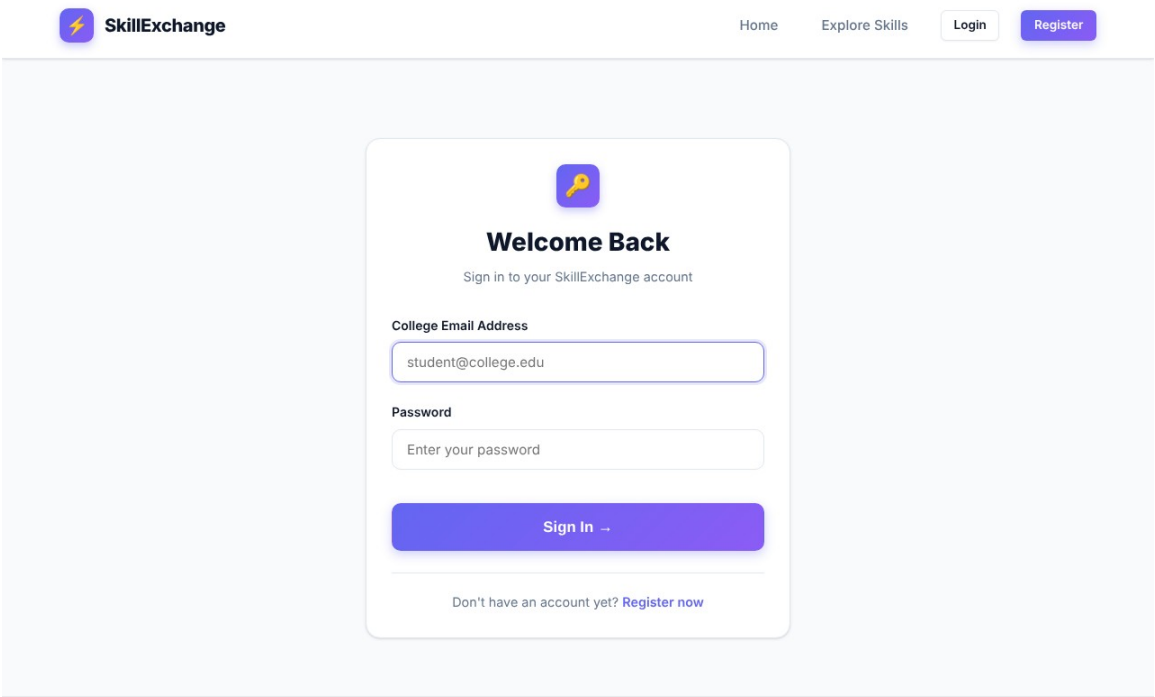


Figure 3.3: Student Login Interface

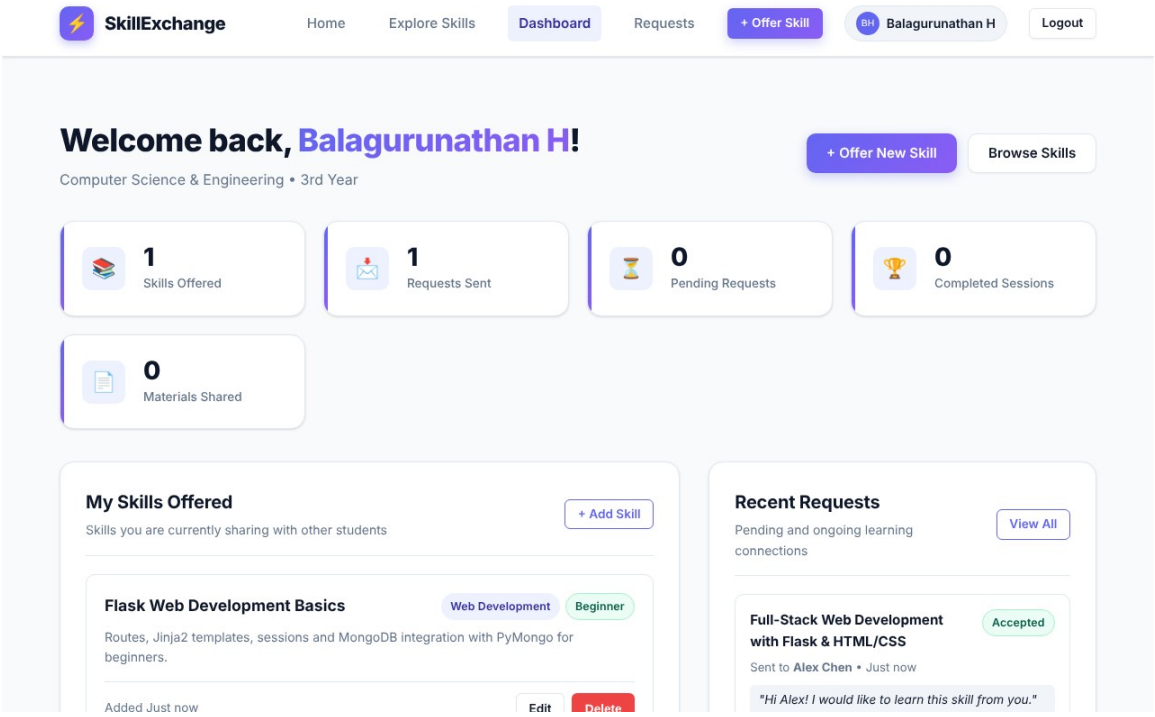


Figure 3.4: Student Dashboard with Statistics and My Skills

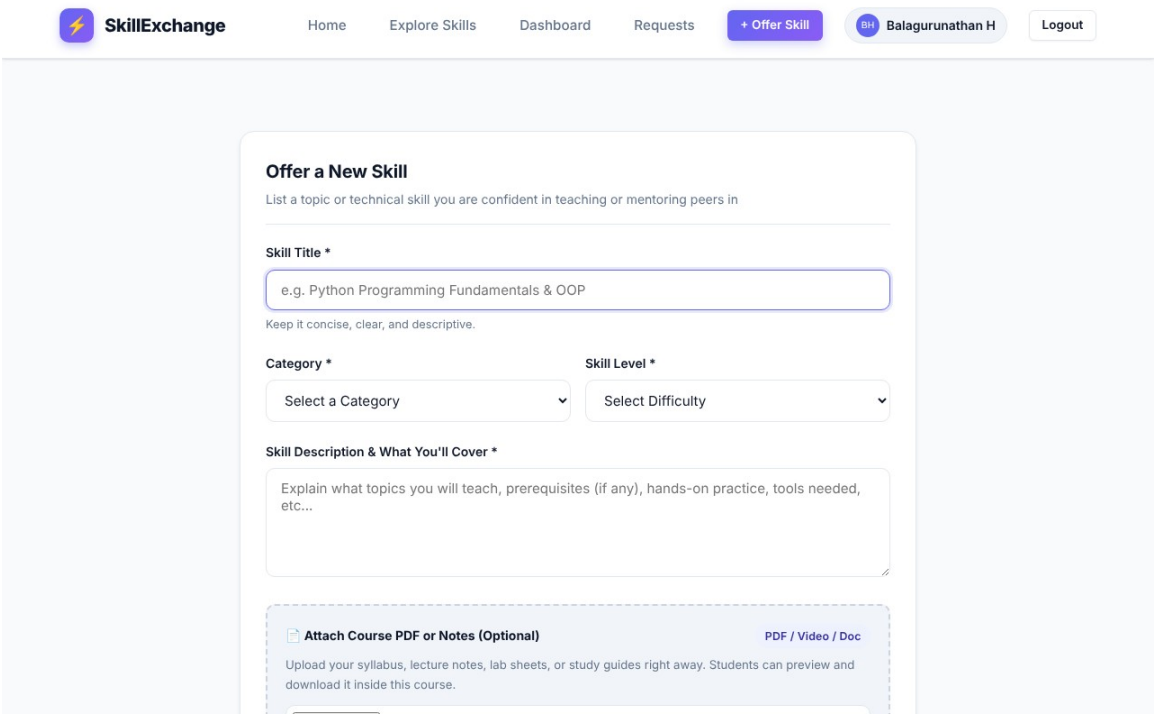


Figure 3.5: Offer a New Skill Form

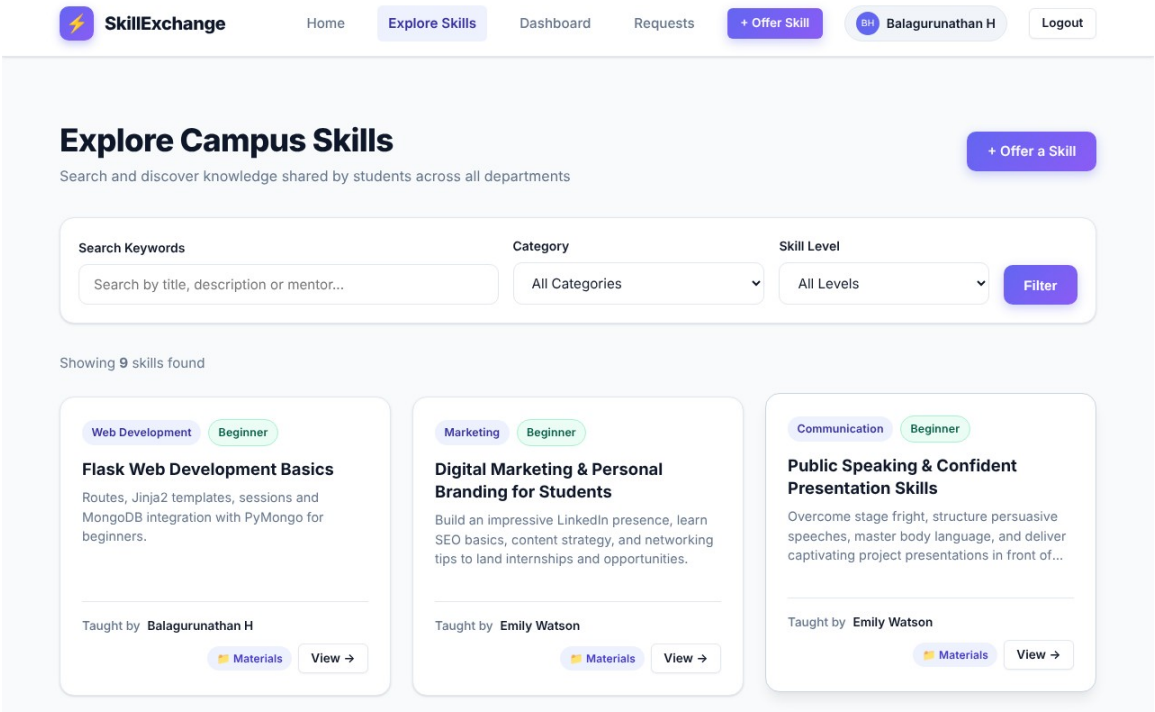


Figure 3.6: Explore Campus Skills Listing

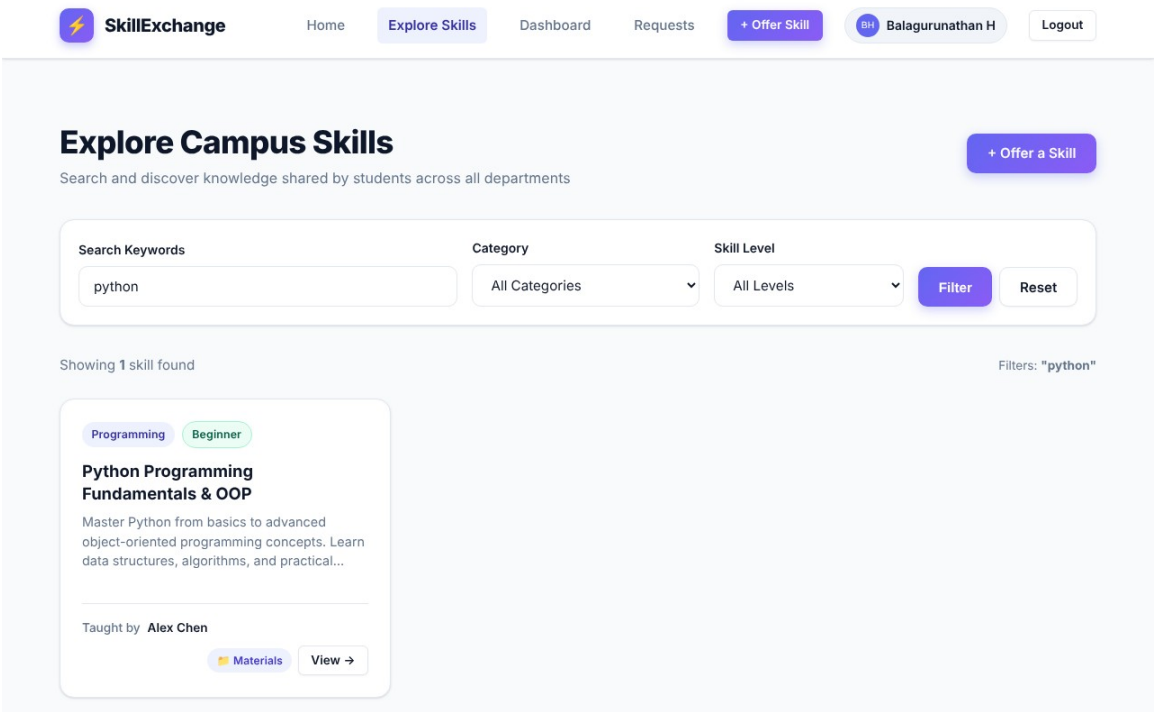


Figure 3.7: Keyword Search Applied to Skills

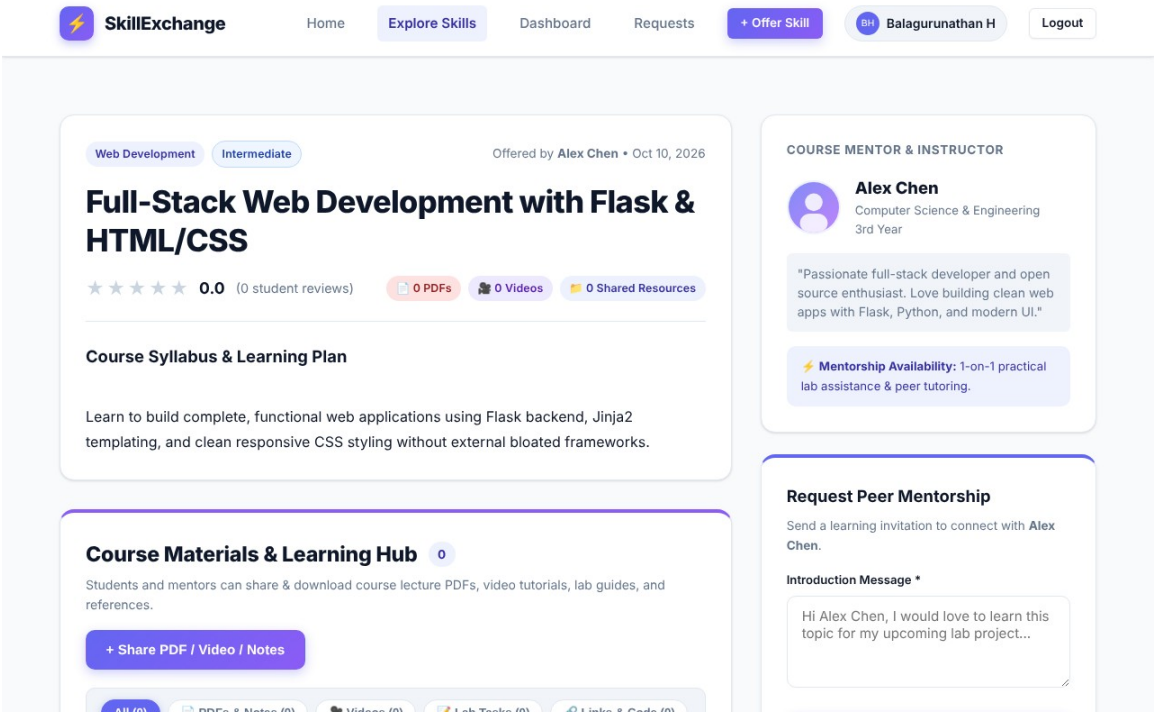


Figure 3.8: Skill Details with Mentor Information and Request Form

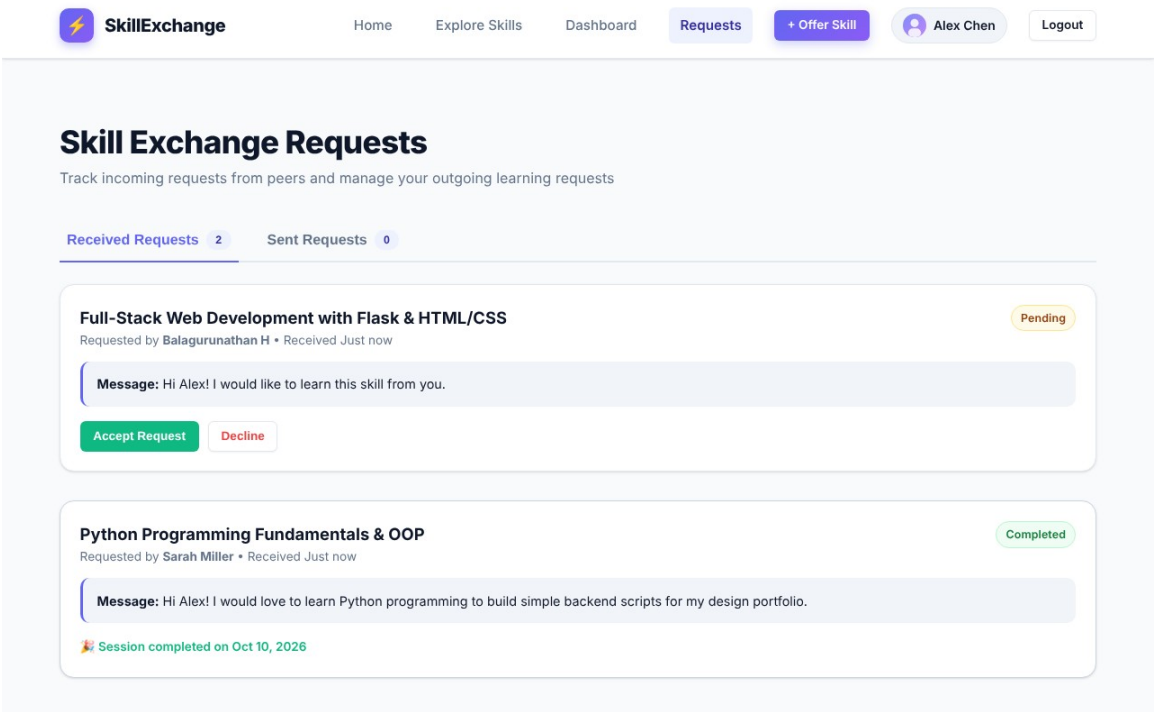


Figure 3.9: Received Requests with Accept and Decline Actions

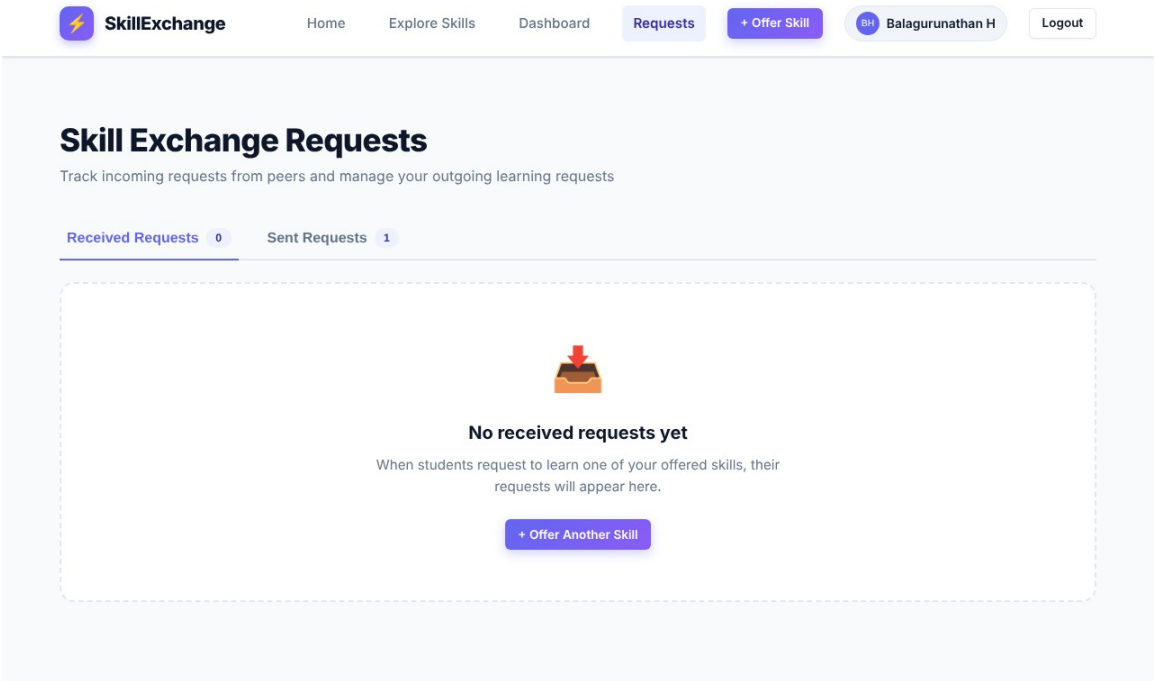


Figure 3.10: Sent Requests Tracking

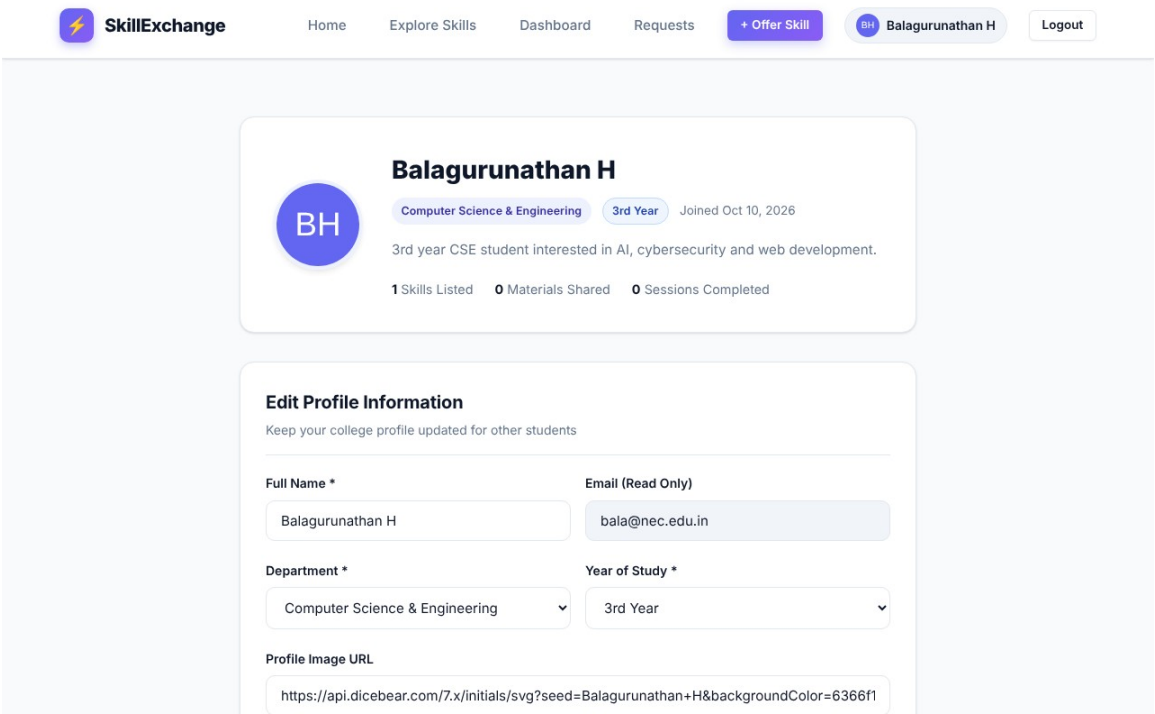


Figure 3.11: Student Profile and Edit Form

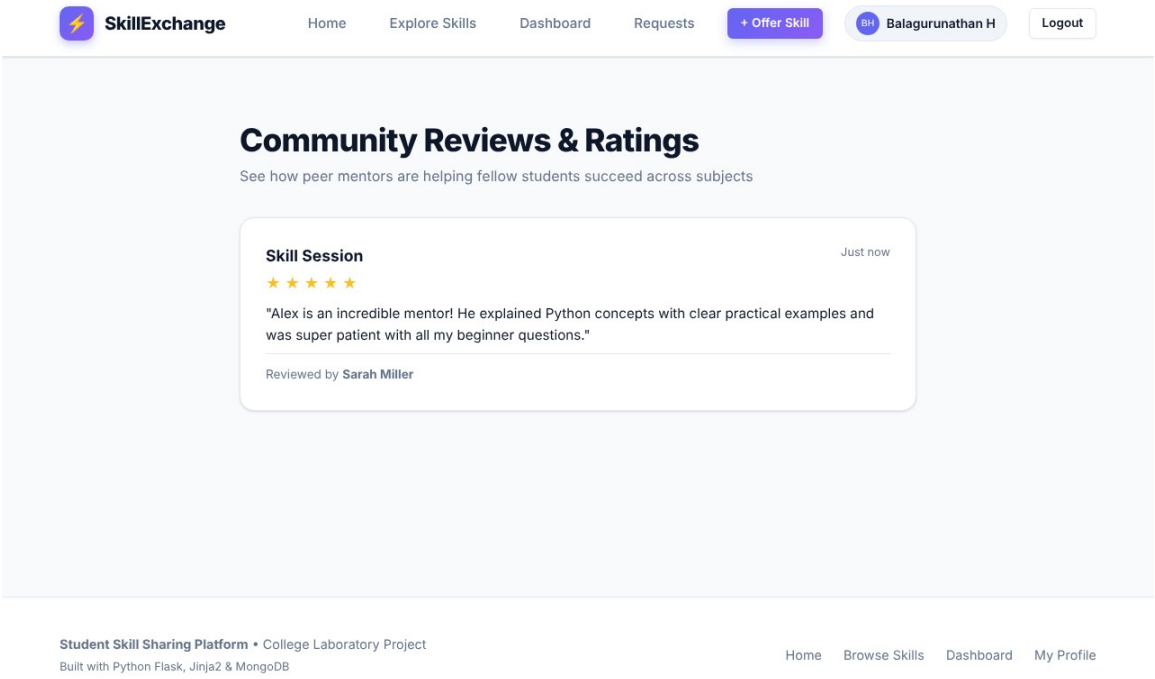


Figure 3.12: Community Reviews and Ratings

CHAPTER 4
CONCLUSION

The Student Skill Exchange Platform has been designed and developed to make peer learning on campus organised, visible and easy to use. Students can register, publish the skills they can teach, search for skills they want to learn, and request sessions from mentors, all within one Flask application backed by MongoDB.

The complete request workflow works end to end. A learning request moves from Pending to Accepted or Rejected, an accepted session is marked Completed, and the learner can then submit a one-to-five star rating with a comment. Validation rules block duplicate requests, self-requests and duplicate reviews, and ownership checks ensure that students can change only their own skills, requests and materials.

The Course Materials Hub adds practical value beyond matching students. PDFs, documents and video links can be shared inside each skill, previewed in the browser, downloaded with usage counts, and upvoted by peers. The dashboard brings together skills offered, requests sent, pending requests, completed sessions and shared materials in a single view.

In conclusion, the platform shows how a server-rendered Flask application with a document database can solve a real campus problem using simple, readable code and no heavy front-end framework. Future work can include real-time notifications, a messaging channel between mentor and learner, session scheduling with calendar support, and a recommendation feature that suggests mentors based on a student's interests.